STUDY NOTE · JAVA / SPRING BOOT FOR PYTHONISTAS

Python 개발자를 위한 Java · Spring Boot 정리노트

prismview-collector를 "이해하면서" 계속 만들기 위한 개념 노트. 지금까지 실제로 겪은 에러를 예시로, FastAPI·SQLAlchemy·Alembic·pytest와 나란히 놓고 설명한다.

Java 21 · Spring Boot 4.1.2 · Gradle (Groovy DSL)
JDBC · PostgreSQL 17 · Flyway · docker compose

CONTENTS

목차

각 장의 끝에 "한 장 요약"이 있다. 질문은 답을 가린 채 먼저 스스로 답해 보고 바로 아래 해설을 확인할 것.

01 **Java는 어떻게 실행되는가** 3
javac · 바이트코드 · JVM · JDK/JRE · main · jar

02 **이 프로젝트를 읽기 위한 최소 Java 문법** 7
class · 정적 타입 · package · 접근 제어자 · 어노테이션 · record · interface

03 **Gradle: 빌드 도구와 프로젝트 구조** 12
build.gradle · gradlew · build / bootRun / test · 표준 폴더 구조

04 **Spring Boot의 핵심: Bean과 의존성 주입** 16
IoC 컨테이너 · @SpringBootApplication · 자동 설정 · FastAPI Depends와 비교

05 **설정과 프로필** 19
application.yml · placeholder · Spring 프로필 · 비밀번호 관리 실무

06 **DB 연결: DataSource · HikariCP · Flyway** 22
SQLAlchemy engine · Alembic과 비교 · 앱이 바로 종료되는 이유

07 **테스트: @SpringBootTest와 프로필** 25
contextLoads · @ActiveProfiles · pytest fixture와 비교 · Testcontainers 예고

08 **다음 단계 지도** 28
V1 마이그레이션 · record Candle · RestClient · JdbcTemplate 배치 upsert

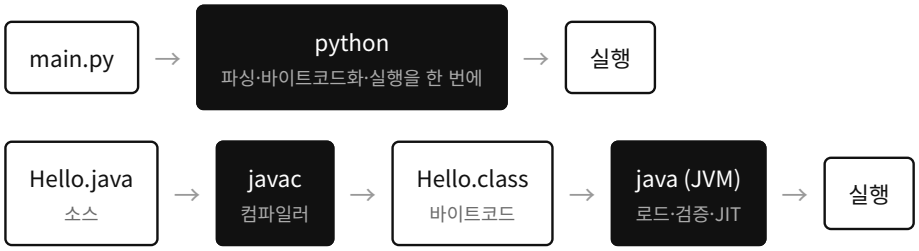
순서를 조금 조정한 이유 — 요청한 1·2장(실행 원리, 문법) 뒤에 **Gradle → Spring Bean → 설정 → DB → 테스트** 순으로 배치했다. 지금까지 겪은 에러 4개가 모두 "Spring이 시작하면서 Bean을 만들다가 설정값이 없어 실패"한 것이라, **Bean이 무엇인지(4장)를 먼저 알아야** 설정(5장)·DB(6장)·테스트(7장) 에러가 한 줄로 꿰어진다.

CHAPTER 01

Java는 어떻게 실행되는가

Python은 소스를 바로 실행하는 것처럼 보이고, Java는 "컴파일"이라는 단계가 눈에 보인다. 이 차이가 이후 모든 도구(Gradle, jar)의 존재 이유다.

1.1 인터프리터 vs 컴파일 + 가상머신



사실 Python도 내부적으로 바이트코드(`__pycache__/*.pyc`)를 만든다. 차이는 **언제, 얼마나 검사하느냐**다.

구분	Python	Java
컴파일 시점	실행할 때 자동으로, 숨겨져 있음	실행 전에 `javac`로 명시적으로
타입 오류 발견	그 줄이 **실행될 때** (mypy를 따로 돌리지 않으면)	**컴파일할 때** — 실행조차 안 됨
결과물	`.pyc` (보통 신경 안 씀)	`.class` → 묶어서 `.jar`
실행 엔진	CPython 인터프리터	JVM. 자주 쓰이는 코드는 JIT이 기계어로 바꿔 빨라짐

Q 1-1 · 생각해 보기

"한 번 작성하면 어디서나 실행(Write once, run anywhere)"이 가능한 이유를, .class 파일이 무엇을 위한 코드인지와 연결해 설명해 보라.

해설 .class 파일은 Intel·ARM 같은 실제 CPU용 기계어가 아니라 **JVM이라는 가상의 CPU용 명령어**다. 맥(M 칩), 리눅스 서버, 윈도우마다 그 OS에 맞는 JVM만 있으면 같은 .class/.jar를 그대로 실행한다. 맥에서 빌드한 jar를 리눅스 서버에 복사해 바로 실행할 수 있는 이유가 이것이다.

1.2 JDK · JRE · JVM, 그리고 왜 JDK를 설치했나

이름	들어 있는 것	Python으로 치면
JVM	바이트코드를 실제로 실행하는 엔진	CPython의 실행 루프
JRE	JVM + 표준 라이브러리 (실행만 가능)	python 실행 파일 + 표준 라이브러리
JDK	JRE + `javac`, `jar`, 디버깅 도구 등 **개발 도구**	위 + 개발 도구 일체

우리는 코드를 **컴파일**해야 하므로 JDK가 필요하다. Homebrew의 `openjdk@21`이 JDK다. (Java 11 이후로는 JRE를 따로 배포하는 경우가 드물어, 사실상 "JDK 하나 설치"가 기본이다.)

JAVA_HOME과 PATH

- **PATH** — 터미널에서 `java`라고 쳤을 때 어느 폴더의 실행 파일을 찾을지. `which python`이 가리키는 것과 같은 원리.
- **JAVA_HOME** — "JDK가 설치된 폴더"를 다른 **도구**(Gradle, IDE)에게 알려주는 변수. Python에는 딱 맞는 대응이 없지만, 가상환경의 `VIRTUAL_ENV`처럼 "기준 위치"를 알려주는 역할이라 보면 된다.

```
# 지금 어떤 Java가 잡혀 있는지 확인 (python --version 대응)
java -version
# Homebrew openjdk@21 경로를 JAVA_HOME으로 지정하는 예 (~/.zshrc)
export JAVA_HOME="$(brew --prefix openjdk@21)/libexec/openjdk.jdk/Contents/Home"
export PATH="$JAVA_HOME/bin:$PATH"
```

Q 1-2 · 생각해 보기

맥에 Java 17과 21이 둘 다 있다. 터미널의 `java -version`은 17인데 프로젝트는 21로 잘 빌드된다. 어떻게 가능할까?

해설 Gradle은 터미널 PATH의 java가 아니라 **JAVA_HOME 또는 build.gradle의 toolchain 설정**(`java { toolchain { languageVersion = JavaLanguageVersion.of(21) } }`)을 보고 JDK를 고른다. start.spring.io가 만든 build.gradle에 toolchain이 들어 있으면, Gradle이 21을 찾아 쓴다. pyenv로 전역 python은 3.10인데 프로젝트 `.python-version`이 3.12인 상황과 비슷하다.

1.3 단일 파일 실행과 main 메서드

PYTHON

```python
def greet(name):
    return f"hi {name}"

if __name__ == "__main__":
    print(greet("minseok"))
```

JAVA · Hello.java

```java
public class Hello {
    static String greet(String name) {
        return "hi " + name;
    }
    public static void main(String[] args) {
        System.out.println(greet("minseok"));
    }
}
```

```
# Java 11+ : 컴파일 단계를 숨기고 소스 파일을 바로 실행 (학습·실험용)
java Hello.java
# 정석: 컴파일 후 실행
javac Hello.java   # → Hello.class 생성
java Hello         # 확장자 없이 "클래스 이름"을 지정
```

Python에서는 파일의 위에서부터 코드가 실행되고, `if __name__ == "__main__"`은 "직접 실행됐을 때만"을 거르는 **관례**다. Java에는 "파일 위에서부터 실행"이라는 개념이 없다. JVM은 지정한 클래스에서 **정확히 이 모양**의 메서드를 찾아 거기서 시작한다.

main 시그니처 조각	의미
`public`	JVM(클래스 바깥)이 호출할 수 있어야 함
`static`	객체를 만들지 않고 클래스 이름으로 바로 호출 (시작 시점엔 객체가 없으니까)
`void`	반환값 없음 (종료 코드는 `System.exit(n)`)
`String[] args`	명령행 인자 = Python의 `sys.argv[1:]`

우리 프로젝트의 `CollectorApplication.java`에도 main이 있다. 그 안의 `SpringApplication.run(...)` 한 줄이 Spring 전체를 띄운다 (4장).

Q 1-3 · 생각해 보기

`./gradlew bootRun --args='--spring.profiles.active=local'`에서 `--spring.profiles.active=local`은 결국 어디로 전달될까?

해설 main의 `String[] args`로 들어가고, main이 그것을 `SpringApplication.run(CollectorApplication.class, args)`에 넘긴다. Spring은 `--키=값` 형태의 인자를 **설정값**으로 해석한다. 즉 명령행 인자 → main → Spring 설정이라는 경로다. uvicorn에 `--port`를 넘기는 것과 비슷하지만, Spring에선 **어떤 설정 키든** 이렇게 덮어쓸 수 있다.

1.4 jar 파일이란

jar(Java ARchive)는 **.class 파일들 + 리소스(yml 등) + 메타정보**를 묶은 zip 파일이다. 실제로 `unzip -l 파일.jar`로 열어볼 수 있다.

종류	설명	Python 대응
라이브러리 jar	다른 프로젝트가 의존성으로 가져다 쓰는 코드 묶음 (예: PostgreSQL 드라이버)	PyPI의 wheel(`.whl`)
실행 가능한 jar (Spring Boot "fat jar")	내 코드 + **모든 의존성 jar**까지 한 파일에. `java -jar collector.jar`만으로 실행	딱 맞는 대응은 없음. "venv까지 통째로 묶은 실행 파일"(PyInstaller 느낌)에 가까움

start.spring.io에서 Packaging=**Jar**를 고른 것이 바로 이 실행 가능한 jar를 만들겠다는 뜻이다. 서버엔 JDK(또는 JRE)와 jar 하나만 있으면 된다 → Docker 이미지도 단순해진다.

Q 1-4 · 생각해 보기

Python 서비스를 서버에 배포할 땐 보통 `pip install -r requirements.txt`를 서버에서 실행한다. Spring Boot fat jar 배포에서는 이 단계가 왜 필요 없을까? 그 대가는?

해설 의존성이 빌드 시점에 이미 jar 안에 들어가 있기 때문이다. 서버에서 패키지 저장소에 접속할 필요가 없어 **재현성**이 높다 (폐쇄망이 많은 금융권에서 특히 유리). 대가는 jar 크기가 커지고(수십 MB), 의존성 하나만 바꿔도 jar 전체를 다시 빌드·배포해야 한다는 점이다.

CHAPTER 01 · 한 장 요약

- `.java` → `javac` → `.class` (JVM용 바이트코드) → `java` (JVM)가 실행. 타입 오류는 **컴파일 때** 잡힌다.
- JDK = 개발 도구 포함 전체. PATH는 터미널용, JAVA_HOME은 Gradle·IDE용 기준점.
- 시작점은 `public static void main(String[] args)` 하나. args → Spring 설정으로 흘러간다.
- jar = .class를 묶은 zip. Spring Boot jar는 의존성까지 포함한 실행 파일.

CHAPTER 02

이 프로젝트를 읽기 위한 최소 Java 문법

문법 전체가 아니라, prismview-collector의 다음 단계 코드를 읽고 쓰는 데 필요한 것만.

2.1 모든 코드는 class 안에 있다

Python 모듈에는 함수·변수·클래스가 파일 최상단에 자유롭게 놓인다. Java에는 **파일 최상단에 함수나 변수가 올 수 없다.** 모든 메서드와 필드는 어떤 class(또는 record, interface, enum) 안에 있어야 한다.

PYTHON · candle_util.py

```python
BAR_SECONDS = 30

def bucket_start(ts_ms: int) -> int:
    size = BAR_SECONDS * 1000
    return ts_ms - ts_ms % size
```

JAVA · CandleUtil.java

```java
public final class CandleUtil {
    public static final int BAR_SECONDS = 30;

    public static long bucketStart(long tsMs) {
        long size = BAR_SECONDS * 1000L;
        return tsMs - tsMs % size;
    }
    private CandleUtil() {} // 객체 생성 금지
}
```

"모듈 수준 함수"가 필요하면 Java에선 **static 메서드만 모은 클래스**로 만든다. 그래서 `Math.max()`, `Collections.sort()` 같은 이름이 생긴다. 또한 **public class 이름 = 파일 이름**이어야 한다 (`CandleUtil` ↔ `CandleUtil.java`).

2.2 정적 타입: 힌트가 아니라 "강제"

상황	Python (타입 힌트)	Java
선언	`price: float = 1.5`	`double price = 1.5;`
잘못된 대입	`price = "abc"` — **그냥 실행됨**	**컴파일 에러**, 실행 불가
함수	`def f(x: int) -> str:`	`String f(int x)` — 반환 타입이 앞에
추론	—	`var list = new ArrayList<String>();` (지역 변수만)
None	`Optional[int]`, `None`	`null`. 단 `int` 같은 원시 타입은 null 불가 → `Integer`

> **금융 데이터 주의** 가격·수량에 `double`을 쓰면 `0.1 + 0.2 != 0.3` 문제가 생긴다(Python float와 동일). Java에서는 `BigDecimal`이 Python `decimal.Decimal`의 대응이다. 캔들 OHLC를 무엇으로 담을지는 8장에서 다시 다룬다.

Q 2-1 · 생각해 보기

`long size = BAR_SECONDS * 1000L;`에서 `1000` 뒤에 붙은 `L`은 왜 있을까? 밀리초 타임스탬프를 다룬다는 점을 떠올려 보라.

해설 `int`는 약 ±21억까지만 담는다. 현재 밀리초 타임스탬프(약 1.7×10^{12})는 int 범위를 넘으므로 `long`(64비트)이 필요하다. `L`은 "이 숫자 리터럴은 long"이라는 표시로, 곱셈이 int로 계산되다 넘치는(overflow) 것을 막는다. Python int는 크기 제한이 없어 이런 걱정이 없었다 — **Java에선 숫자 타입 크기가 버그의 원인**이 될 수 있다.

2.3 package와 import: 폴더 = 패키지 이름

```java
// src/main/java/com/prismview/collector/CollectorApplication.java
package com.prismview.collector;          // ← 폴더 경로와 반드시 일치

import org.springframework.boot.SpringApplication;  // 다른 패키지의 클래스
```

개념	Python	Java
이름 결정	파일 위치가 곧 모듈 경로 (`app/services/x.py` → `app.services.x`)	파일 맨 위 `package` 선언 + 그 경로와 같은 폴더
import 단위	모듈 또는 그 안의 이름	**클래스** 단위 (`import a.b.ClassName;`)
같은 패키지	상대 import 필요	import 없이 바로 사용
이름 규칙	자유	도메인 역순 (`com.prismview`) — 전 세계에서 이름 충돌 방지

Group=`com.prismview`, Artifact=`collector`로 만들었기 때문에 패키지가 `com.prismview.collector`가 되었다.

Q 2-2 · 생각해 보기

앞으로 `Candle`을 `com/prismview/collector/domain/` 폴더에 만든다. 파일 첫 줄은 무엇이어야 하고, `CollectorApplication`보다 상위 폴더(예: `com/prismview/`)에 Spring 컴포넌트를 두면 왜 문제가 될까? (힌트: 4장의 컴포넌트 스캔)

해설 첫 줄은 `package com.prismview.collector.domain;`. Spring Boot는 `@SpringBootApplication`이 붙은 클래스의 패키지와 **그 하위 패키지만** 스캔해 Bean을 찾는다. 상위나 옆 패키지에 둔 `@Component`는 발견되지 않아 "Bean을 찾을 수 없다" 에러가 난다. 그래서 메인 클래스는 항상 패키지 트리의 **꼭대기**에 둔다.

2.4 접근 제어자와 static

키워드	의미	Python 대응
`public`	어디서나 접근	일반 이름
`private`	같은 클래스 안에서만 — **컴파일러가 막음**	`_name` 관례 (막지는 않음)
(생략)	같은 패키지 안에서만 (package-private)	없음
`protected`	같은 패키지 + 자식 클래스	없음
`static`	객체가 아니라 **클래스 자체**에 속함	클래스 변수, `@staticmethod`
`final`	변수: 재대입 불가 / 클래스: 상속 불가	`typing.Final` (강제 아님)

2.5 어노테이션(@) vs 데코레이터 — 생김새만 같다

이 장에서 가장 중요한 차이다. Spring은 어노테이션 투성이인데, 데코레이터처럼 생각하면 헷갈린다.

	Python 데코레이터	Java 어노테이션
정체	**함수**. 정의 시점에 실행되어 대상을 감싼(바꾼) 결과를 돌려줌	**메타데이터(라벨)**. 그 자체로는 아무 동작도 안 함
누가 동작시키나	데코레이터 자신	그 라벨을 **읽는 누군가**(Spring, JUnit, 컴파일러)
언제	모듈 로드 시	컴파일 시(`@Override`) 또는 실행 중 리플렉션으로 읽을 때
예	`@app.get("/")`가 라우터에 직접 등록	`@Component`를 Spring이 스캔하다 발견하고 Bean 생성

Q 2-3 · 생각해 보기

Spring 없이 `public static void main`에서 `new MyService()`로 객체를 만들면, 클래스에 붙은 `@Component`는 어떤 효과가 있을까?

해설 **아무 효과도 없다.** 어노테이션은 "이 클래스는 컴포넌트다"라는 라벨일 뿐이고, 그 라벨을 읽고 객체를 만들어 관리하는 건 Spring 컨테이너다. 직접 `new`로 만든 객체는 Spring이 모르는 객체라 주입(@Autowired 등)도 일어나지 않는다. 데코레이터라면 정의만 해도 동작했을 것이라는 점과 대조된다.

2.6 record — 다음 단계의 Candle

PYTHON

```python
from dataclasses import dataclass
from decimal import Decimal

@dataclass(frozen=True)
class Candle:
    inst_id: str
    ts: int
    open: Decimal
    high: Decimal
    low: Decimal
    close: Decimal
    volume: Decimal
```

JAVA (16+)

```java
public record Candle(
    String instId,
    long ts,
    BigDecimal open,
    BigDecimal high,
    BigDecimal low,
    BigDecimal close,
    BigDecimal volume
) {}
// 자동 생성: 생성자, instId()·ts() 접근자,
// equals / hashCode / toString
// 모든 필드 final → 불변
```

접근자는 `getTs()`가 아니라 `c.ts()`다(필드 이름 그대로). 값을 바꾸려면 새 record를 만든다 — `dataclasses.replace()`와 같은 사고방식.

```java
public record Candle(...) {
    public Candle {                       // "compact 생성자" = __post_init__ 대응
        if (high.compareTo(low) < 0)
            throw new IllegalArgumentException("high < low");
    }
}
```

Q 2-4 · 생각해 보기

수집기는 여러 스레드가 같은 캔들 데이터를 읽을 수 있다. Candle을 "값을 바꿀 수 있는 일반 클래스"가 아니라 불변 record로 만들면 어떤 이점이 있을까?

해설 불변 객체는 생성 후 바뀌지 않으므로 여러 스레드가 동시에 읽어도 **락 없이 안전**하다. 또한 "1초봉 집계 중에 누가 이미 저장한 캔들을 수정했다" 같은 버그가 원천 차단된다. 집계 중 값이 계속 바뀌는 "진행 중인 30초봉"은 별도의 가변 누적기로 두고, 완성되면 불변 Candle로 **확정**하는 설계가 자연스럽다.

2.7 interface — 구현이 아니라 "약속"

PYTHON · Protocol

```python
from typing import Protocol

class CandleRepository(Protocol):
    def upsert_all(
        self, candles: list[Candle]
    ) -> None: ...
```

JAVA

```java
public interface CandleRepository {
    void upsertAll(List<Candle> candles);
}

public class JdbcCandleRepository
        implements CandleRepository {
    @Override
    public void upsertAll(List<Candle> c) {...}
}
```

Python `Protocol`은 구조적 타이핑(메서드만 맞으면 OK)이고, Java interface는 `implements`로 **명시적으로 선언**해야 한다. `@Override`는 "부모/인터페이스의 메서드를 구현한다"를 컴파일러에게 확인시키는 어노테이션 — 오타로 이름이 다르면 컴파일 에러를 내 준다 (2.5의 "컴파일 시 읽는 어노테이션"의 예).

`List<Candle>`의 꺾쇠는 **제네릭**으로, Python `list[Candle]`과 같은 뜻이지만 역시 컴파일러가 강제한다.

Q 2-5 · 생각해 보기

수집 로직(`CollectorService`)이 `JdbcCandleRepository`가 아니라 `CandleRepository` 인터페이스 타입에 의존하게 만들면, 테스트와 이후 확장(예: Parquet 저장)에서 무엇이 쉬워질까?

해설 테스트에선 DB 없이 리스트에 쌓기만 하는 가짜 구현(InMemoryCandleRepository)을 끼워 수집·집계 로직만 검증할 수 있다. 저장소를 Parquet나 다른 DB로 바꿀 때도 서비스 코드는 그대로 두고 구현체만 교체하면 된다. FastAPI에서 `Depends`로 의존성을 주입하고 테스트에서 `app.dependency_overrides`로 바꿔치기하던 것과 같은 목적이다 — 4장의 DI가 이것을 자동화한다.

CHAPTER 02 · 한 장 요약

- 모든 코드는 class 안. public class 이름 = 파일 이름. 모듈 함수 대신 static 메서드.
- 타입은 강제다. 타임스탬프는 `long`, 금액은 `BigDecimal`.
- `package` 선언 = 폴더 경로. 메인 클래스는 패키지 트리 꼭대기에.
- **어노테이션은 라벨이다.** 읽는 쪽(Spring·컴파일러)이 있어야 동작한다.
- record = `dataclass(frozen=True)`, interface = 명시적 Protocol.

CHAPTER 03

Gradle: 빌드 도구와 프로젝트 구조

Python에서 pip + venv + pytest 실행 + 패키징이 각각 따로였다면, Gradle은 이것을 하나의 도구로 묶는다.

3.1 파일별 역할 대응표

파일	역할	Python 대응
`build.gradle`	의존성, 플러그인, Java 버전, 태스크 설정	`pyproject.toml` (+ requirements.txt)
`settings.gradle`	프로젝트 이름, (멀티모듈이면) 하위 모듈 목록	pyproject의 `[project] name`
`gradlew`, `gradle/wrapper/`	**정해진 버전의 Gradle을 자동으로 받아 실행**하는 스크립트	대응 없음 (poetry 버전 고정과 비슷)
`~/.gradle/caches`	다운로드한 의존성 jar 캐시	pip 캐시 / site-packages
`build/`	컴파일 결과(.class), 테스트 리포트, jar	`__pycache__`, `dist/`

Q 3-1 · 생각해 보기

Gradle을 맥에 따로 설치하지 않았는데 `./gradlew build` 가 동작한다. gradlew와 gradle/wrapper/를 git에 커밋해야 하는 이유는?

해설 gradlew는 `gradle/wrapper/gradle-wrapper.properties`에 적힌 버전의 Gradle을 처음 실행 시 내려받는다. 이걸 커밋하면 팀원·CI 서버·미래의 나 모두 **같은 Gradle 버전**으로 빌드하게 된다("내 PC에선 되는데" 방지). Python에서 poetry.lock을 커밋하는 이유와 같은 재현성 문제다. 참고로 Gradle은 의존성을 공용 캐시에 받아 두고 프로젝트마다 build.gradle 기준으로 골라 쓰기 때문에, venv 같은 별도 격리 단계가 필요 없다.

3.2 build.gradle 읽기

```
plugins {
    id 'java'
    id 'org.springframework.boot' version '4.1.2'          // bootRun, bootJar 태스크 제공
    id 'io.spring.dependency-management' version '...'     // Spring 호환 버전 자동 지정
}
group = 'com.prismview'
java { toolchain { languageVersion = JavaLanguageVersion.of(21) } }

dependencies {
    implementation '...jdbc 스타터...'         // 컴파일 + 실행에 필요
    implementation '...flyway...'
    runtimeOnly 'org.postgresql:postgresql'    // 실행 때만 (코드에서 직접 import 안 함)
    testImplementation '...test 스타터...'     // 테스트에서만 = dev 의존성
}
```

버전이 없는 줄이 많다. **dependency-management 플러그인이 Spring Boot 4.1.2와 호환되는 버전을 일괄 지정**하기 때문이다. Python이라면 "FastAPI 0.x와 맞는 pydantic·starlette 버전"을 직접 맞추던 일을 대신해 준다.

구성(configuration)	의미	Python 대응
`implementation`	내 코드가 직접 사용	일반 dependencies
`runtimeOnly`	실행할 때만 필요. JDBC 드라이버가 대표적	— (psycopg를 URL로만 쓰는 상황)
`testImplementation`	테스트 코드에서만	`[dev]` / dev-dependencies

Q 3-2 · 생각해 보기

PostgreSQL 드라이버가 `runtimeOnly`여도 되는 이유는? 우리 코드는 PostgreSQL 클래스를 어떻게 사용하게 되나?

해설 우리 코드는 `org.postgresql...` 클래스를 직접 import하지 않고, 표준 인터페이스인 **JDBC**(`javax.sql.DataSource`, `JdbcTemplate`)만 쓴다. 실행 시 `jdbc:postgresql://` URL을 보고 드라이버가 연결을 담당한다. SQLAlchemy에서 `postgresql+psycopg://` URL만 쓰고 psycopg를 직접 import하지 않던 것과 같다. 덕분에 컴파일 시점에 특정 DB에 묶이지 않는다.

3.3 주요 태스크

명령	하는 일	Python 대응
`./gradlew compileJava`	main 소스 컴파일	(없음 / mypy 비슷)
`./gradlew test`	테스트 컴파일 + 실행	`pytest`
`./gradlew build`	컴파일 → **테스트** → jar 생성	pytest && python -m build
`./gradlew build -x test`	테스트를 **제외**(`-x` = exclude)하고 빌드	—
`./gradlew bootRun`	jar 없이 바로 앱 실행	`uvicorn main:app`
`./gradlew clean`	`build/` 삭제	rm -rf dist

실제로 겪은 일 **#1** · build는 성공, 테스트 포함 build는 실패

```
./gradlew build -x test   # BUILD SUCCESSFUL
./gradlew build           # contextLoads() FAILED — DataSourceBeanCreationException
```

이 결과에서 알 수 있는 것: **코드는 문법적으로 멀쩡하다**(컴파일·jar 생성 성공). 실패는 "테스트가 Spring을 실제로 띄워보는 과정"에서 났다. 즉 **컴파일 에러가 아니라 실행 시점(런타임) 설정 에러**다. 원인과 해결은 5·7장에서 이어진다.

Q 3-3 · 생각해 보기

`-x test`로 테스트를 건너뛰면 빌드가 성공한다. 그럼 CI에도 `-x test`를 넣으면 될까?

해설 안 된다. 그건 화재경보기 배터리를 빼고 "경보가 안 울린다"고 하는 것과 같다. contextLoads 실패는 실제로 "이 설정이면 앱이 시작조차 못 한다"는 **진짜 신호**였다 (#2에서 bootRun도 같은 이유로 실패). `-x test`는 원인을 분리해서 볼 때 잠깐 쓰는 진단 도구다.

3.4 표준 폴더 구조 (Maven 관례)

```
src/
 ├─ main/java/        ← 운영 코드 (.java)        → 컴파일되어 jar에 들어감
 ├─ main/resources/   ← 설정·SQL 등 (yml, db/migration/*.sql) → 그대로 jar에 복사
 └─ test/java/        ← 테스트 코드              → jar에 안 들어감
```

Python은 폴더 구조가 자유롭지만 Java 진영은 이 구조가 사실상 **표준**이라, 설정을 따로 하지 않아도 Gradle이 어디에 무엇이 있는지 안다("설정보다 관례"). Flyway가 `src/main/resources/db/migration/`을 기본 위치로 보는 것도 같은 철학이다 (6장).

CHAPTER 03 · 한 장 요약

- build.gradle = pyproject.toml. gradlew = 버전 고정된 Gradle 실행기(커밋 필수).
- implementation / runtimeOnly / testImplementation으로 의존성의 쓰임새를 구분.
- `build` = 컴파일 + **테스트** + jar. `-x test` 성공 + 일반 build 실패 = 런타임 설정 문제.
- src/main/java · src/main/resources · src/test/java는 관례이자 표준.

CHAPTER 04

Spring Boot의 핵심: Bean과 의존성 주입

지금까지의 에러 이름에 모두 "Bean"이 들어 있었다. Spring을 이해하는 열쇠는 하나 — "객체를 내가 만들지 않고 컨테이너가 만든다."

4.1 IoC 컨테이너와 Bean

Bean = Spring 컨테이너(ApplicationContext)가 만들고 관리하는 객체. **IoC(제어의 역전)** = 객체 생성과 연결을 내 코드가 아니라 컨테이너가 한다는 뜻.

PYTHON · 직접 조립

```python
engine = create_engine(DB_URL)
repo = CandleRepository(engine)
client = OkxClient(httpx.Client())
service = CollectorService(client, repo)
service.run()
```

JAVA · 컨테이너가 조립

```java
@Service
public class CollectorService {
    private final OkxClient client;
    private final CandleRepository repo;

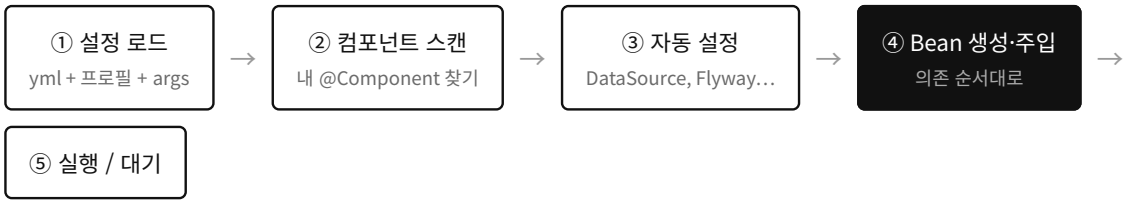
    // 생성자 주입: Spring이 맞는 Bean을 찾아 넣어 줌
    public CollectorService(OkxClient client,
                            CandleRepository repo) {
        this.client = client; this.repo = repo;
    }
}
```

FastAPI의 `Depends(get_db)`와 목적이 같다: 사용하는 쪽은 "무엇이 필요한지"만 선언하고, 만드는 책임은 바깥에 둔다. 차이는 FastAPI의 Depends는 **요청마다** 실행되고, Spring Bean은 기본적으로 **앱 시작 시 한 번 만들어 공유(싱글톤)** 된다는 점이다.

Bean을 등록하는 방법	설명
`@Component` / `@Service` / `@Repository`	클래스에 라벨 → 컴포넌트 스캔이 발견해 생성. 셋은 역할 표시만 다르다
`@Configuration` + `@Bean` 메서드	외부 라이브러리 객체처럼 클래스에 라벨을 못 붙일 때, 메서드 반환값을 Bean으로 등록
자동 설정(Auto-configuration)	Spring Boot가 의존성을 보고 알아서 등록. `DataSource`, `JdbcTemplate`, `Flyway`가 이것

4.2 @SpringBootApplication과 시작 과정

```
@SpringBootApplication                    // = @Configuration + @EnableAutoConfiguration +
@ComponentScan
public class CollectorApplication {
    public static void main(String[] args) {
        SpringApplication.run(CollectorApplication.class, args);
    }
}
```



④에서 하나라도 실패하면 **앱 전체가 시작되지 않는다.** FastAPI는 DB 설정이 틀려도 서버는 뜨고 첫 요청에서 에러가 나는 경우가 많지만, Spring은 시작 시점에 미리 모든 Bean을 만들어 보므로 **"빨리, 시끄럽게 실패(fail-fast)"**한다.

Q 4-1 · 생각해 보기

우리 코드에는 DataSource를 만드는 코드가 한 줄도 없다. 그런데 에러 이름은 `DataSourceBeanCreationException`이었다. 누가 DataSource Bean을 만들려고 했을까?

해설 ③ 자동 설정이다. Spring Boot는 클래스패스에 JDBC 스타터와 PostgreSQL 드라이버가 있는 것을 보고 "DB를 쓰겠구나"라고 판단해 **HikariCP 기반 DataSource Bean을 자동으로 만들려고 시도**한다. 그 재료(url, username, password)를 설정에서 읽는데, 값이 없거나 잘못되어 실패한 것이다. 의존성을 추가한 것만으로 "DB 연결이 필수인 앱"이 된 셈이다.

Q 4-2 · 생각해 보기

Bean이 싱글톤이라면, CollectorService 안에 "지금 집계 중인 30초봉"을 필드로 두는 것은 안전할까?

해설 하나의 객체를 모든 곳에서 공유하므로, 필드에 상태를 두면 **모든 사용자(스레드)가 그 상태를 공유**한다. 수집 스레드가 하나뿐이면 문제가 드러나지 않을 수 있지만, 여러 종목(BTC/ETH)을 병렬로 수집하면 서로 덮어쓸 수 있다. Spring Bean은 보통 **상태 없는(stateless)** 서비스로 만들고, 상태가 필요하면 종목별 Map을 `ConcurrentHashMap`으로 두는 등 동시성을 명시적으로 설계한다.

4.3 왜 필드 주입이 아니라 생성자 주입인가

예전 코드에서 흔한 `@Autowired private OkxClient client;`(필드 주입) 대신 생성자 주입이 권장된다.

- `final`로 선언 가능 → 한 번 주입되면 바뀌지 않음 (불변)
- 테스트에서 Spring 없이 `new CollectorService(fakeClient, fakeRepo)`로 바로 생성 가능
- 생성자가 하나면 `@Autowired`도 생략 가능

CHAPTER 04 · 한 장 요약

- Bean = 컨테이너가 만들고 관리하는 (기본 싱글톤) 객체. Depends의 앱 전체 버전.
- @SpringBootApplication = 설정 + 자동 설정 + 컴포넌트 스캔.
- 의존성만 추가해도 자동 설정이 DataSource·Flyway Bean을 만들려 한다 → 설정값 필수.
- Spring은 시작 시점에 모든 Bean을 만들어 보고, 하나라도 실패하면 **fail-fast**.

CHAPTER 05

설정과 프로필

python-dotenv와 os.getenv에 익숙하다면, Spring의 설정은 "여러 출처를 우선순위대로 겹쳐 하나의 설정 공간을 만든다"로 이해하면 된다.

5.1 설정이 들어오는 여러 출처

Spring은 여러 곳의 설정을 하나의 **Environment**로 합친다. 같은 키가 여러 곳에 있으면 **우선순위가 높은 것이 이긴다**. (아래는 자주 쓰는 것만, 위가 높음)

순위	출처	예
1	명령행 인자	`--spring.profiles.active=local`
2	OS 환경변수	`DB_URL=... ./gradlew bootRun`
3	프로필별 파일	`application-local.yml`
4	기본 파일	`application.yml`

환경변수는 **느슨한 바인딩**(relaxed binding) 덕분에 `SPRING_DATASOURCE_URL` 처럼 대문자·밑줄로 써도 `spring.datasource.url`에 연결된다. Docker·K8s에서 환경변수로 설정을 덮어쓰기 쉬운 이유다.

5.2 placeholder `${...}`

PYTHON

```
url = os.getenv("DB_URL")              # 없으면 None
url = os.getenv("DB_URL", "default")   # 기본값
url = os.environ["DB_URL"]             # 없으면 KeyEr
```

SPRING YAML

```
url: ${DB_URL}            # 다른 설정 키/환경변수에서
url: ${DB_URL:default}    # 기본값
url: ${DB_URL:}           # 빈 문자열 기본값 (주의!)
```

`${DB_URL}`은 "**DB_URL이라는 이름의 설정값**을 가져와라"다. 그 값은 환경변수일 수도, 다른 yml 파일의 키일 수도 있다. 그래서 우리 구조가 동작한다:

```
# application.yml (공통, 커밋) — 무엇이 필요한지만
spring:
  datasource:
    url: ${DB_URL}
    username: ${DB_USERNAME}
    password: ${DB_PASSWORD}

# application-local.yml (내 Docker DB 전용) — 최상위 키로 값 제공
DB_URL: jdbc:postgresql://localhost:5432/prismview?reWriteBatchedInserts=true
DB_USERNAME: prismview
DB_PASSWORD: prismview
```

실제로 겪은 일 **#2** · 프로필 없이 bootRun

```
No active profile set, falling back to 1 default profile: "default"
...  'url' must start with "jdbc"
```

프로필을 지정하지 않으니 `application-local.yml` 은 **읽히지 않았다.** `DB_URL` 을 제공하는 곳이 어디에도 없어 placeholder가 풀리지 않았고, 결과적으로 `${DB_URL}` 이라는 **문자열 그대로**가 url로 들어갔다. HikariCP는 "jdbc:로 시작하지 않는 URL"이라며 거부했다. 에러 메시지가 원인(프로필 미지정)이 아니라 **증상**(이상한 URL)을 말하고 있다는 점이 핵심이다.

Q 5-1 · 생각해 보기

블로그 노트에 적은 실수 중 `${DB_PASSWORD:}`(빈 기본값)이 있었다. `${DB_PASSWORD}` 보다 편해 보이는데 왜 위험할까?

해설 빈 기본값을 주면 값이 없어도 placeholder 해석이 **"성공"**한다. 앱은 비밀번호 빈 문자열로 DB에 접속을 시도하고, 한참 뒤 "인증 실패"라는 엉뚱한 곳에서 에러가 난다. 기본값이 없으면 설정 누락이 시작 단계에서 드러난다. 필수 값은 기본값을 두지 않는 것이 fail-fast 원칙에 맞다. Python에서 `os.getenv("X", "")` 보다 `os.environ["X"]` 가 나은 상황과 같다.

5.3 Spring 프로필

프로필 = **환경 이름**. `local` 을 활성화하면 `application.yml` 위에 `application-local.yml` 이 **덮어써지듯 합쳐진다**.

지정 방법	예
실행 인자	`./gradlew bootRun --args='--spring.profiles.active=local'`
환경변수	`SPRING_PROFILES_ACTIVE=local ./gradlew bootRun`
jar 실행	`java -jar collector.jar --spring.profiles.active=prod`
테스트	`@ActiveProfiles("local")` (7장)

Q 5-2 · 생각해 보기

매번 `--args` 를 치기 귀찮아 `application.yml` 에 `spring.profiles.active: local` 을 박아두고 싶다. 그러면 무엇이 위험해질까?

해설 그 jar를 운영 서버에 그대로 올리면 **운영에서도 local 프로필**이 기본으로 켜진다. 운영 설정이 빠졌을 때 에러로 멈추는 대신 엉뚱한(로컬) 값으로 조용히 동작할 수 있다. 그래서 실무에선 기본 프로필을 파일에 고정하기보다 **실행 시점에 지정**하는 경우가 많다. 로컬 편의를 원하면 IntelliJ 실행 설정이나 셸 alias처럼 **내 PC에만 있는 곳**에 두는 편이 안전하다.

5.4 금융권 실무에서는 비밀번호를 어떻게 다루나

> **원칙: 비밀번호는 코드와 레포에 두지 않고, 환경별로 분리하며, 운영 비밀번호는 개발자가 모르게 한다.**

방식	내용	어디서 흔한가
Spring 프로필	환경별 yml 분리, 운영 yml에도 비밀번호는 안 씀	거의 기본
WAS + JNDI	JEUS·WebLogic 등에 운영팀이 DB 연결을 설정, 앱은 `jdbc/xxxDS` 이름으로 빌려 씀	전통 금융권 시스템
설정값 암호화	`ENC(암호문)` + 실행 시 복호화 키 주입 (Jasypt 등)	국내 SI·금융 프로젝트
비밀 관리 시스템	Vault, AWS Secrets Manager, K8s Secret — 조회·교체·접근 기록	클라우드·최신 시스템
`.env`	Node·Python 문화에 가까움. 로컬 개발용으로 쓰는 팀도 있음	금융권 Java 표준은 아님

금융권이 특히 엄격한 이유는 **감사**(누가 언제 운영 DB에 접근했는지 추적), **직무 분리**(개발자가 만들고 운영자가 배포·관리), **평문 저장 금지** 원칙 때문이다. 다만 회사·기관마다 실제 방식은 다르므로 "흔한 패턴" 정도로 이해하자.

Q 5-3 · 생각해 보기

로컬 Docker DB 비밀번호 `prismview`를 `application-local.yml`에 적어 커밋해도 괜찮은 이유는? 반대로 절대 커밋하면 안 되는 것은?

해설 `prismview`는 내 맥의 Docker 안에서만 쓰이는 값이라, 노출돼도 그걸로 접근할 수 있는 대상이 없다. 반면 운영 DB 비밀번호, 다른 서비스에서 재사용하는 개인 비밀번호, API 키(OKX 개인 키 등)는 노출되는 순간 실제 피해로 이어진다. 판단 기준은 **"이 값이 새면 무엇에 접근할 수 있는가"**다. 그리고 지금은 로컬 전용이라도, 나중에 이 파일에 진짜 키를 추가하는 실수를 막으려면 OKX 키는 처음부터 환경변수로만 받도록 설계해 두는 것이 좋다.

CHAPTER 05 · 한 장 요약

- 명령행 인자 > 환경변수 > 프로필 yml > 기본 yml 순으로 덮어쓴다.
- `${KEY}` 는 "설정 공간의 KEY"를 참조. 해결 안 되면 증상이 엉뚱한 곳에서 터진다(#2).
- 필수 값엔 기본값을 두지 말 것 — fail-fast.
- 프로필은 실행 시점에 지정. 운영 비밀번호는 개발자도 모르게(JNDI, Vault 등).

CHAPTER 06

DB 연결: DataSource · HikariCP · Flyway

SQLAlchemy의 engine과 Alembic을 알고 있다면 이미 절반은 안다. 이름과 시작 시점이 다를 뿐이다.

6.1 계층 대응표

역할	Python 생태계	이 프로젝트 (Java)
DB 드라이버	psycopg	PostgreSQL JDBC Driver
표준 인터페이스	DB-API 2.0 (PEP 249)	**JDBC**
커넥션 풀	`create_engine()` 내부의 QueuePool	**HikariCP** (`DataSource`)
SQL 실행 도우미	SQLAlchemy Core `conn.execute(text(...))`	**JdbcTemplate**
ORM	SQLAlchemy ORM	JPA/Hibernate (이번엔 안 씀)
마이그레이션	Alembic	**Flyway**

Q 6-1 · 생각해 보기

이 프로젝트는 ORM(JPA)이 아니라 JdbcTemplate을 골랐다. 30초봉을 대량으로 계속 쌓는 수집기에서 이 선택이 맞는 이유는?

해설 수집기는 복잡한 객체 관계가 없고, "같은 모양의 행을 대량으로 빠르게 넣고, 중복이면 갱신"이 전부다. 이 작업은 PostgreSQL의 `INSERT ... ON CONFLICT DO UPDATE`를 직접 쓰는 게 가장 명확하고 빠르다. ORM은 객체 상태 추적(영속성 컨텍스트) 같은 기능이 오히려 오버헤드가 된다. SQLAlchemy에서도 대량 적재는 ORM 대신 Core의 `insert().on_conflict_do_update()`를 쓰던 것과 같은 판단이다.

6.2 DB URL 해부

```
jdbc:postgresql://localhost:5432/prismview?reWriteBatchedInserts=true
└─┘ └────────┘   └───────┘ └─┘ └──────┘ └──────────────────────────┘
 |    드라이버      호스트    포트  DB 이름       드라이버 옵션
 └ JDBC URL이라는 표시 (이게 없어서 #2 에러)
```

- SQLAlchemy URL(`postgresql+psycopg://user:pw@host/db`)과 달리 사용자·비밀번호는 보통 **URL 밖**(username/password 설정)에 둔다.
- `reWriteBatchedInserts=true`: 배치 INSERT 여러 개를 드라이버가 multi-row INSERT로 재작성 → 대량 적재 성능 향상. 8장의 배치 upsert를 위한 사전 준비다.

Q 6-2 · 생각해 보기

docker-compose.yml의 `ports: "5432:5432"`와 URL의 `localhost:5432`는 무엇이 다를까? 호스트 자리에 서비스 이름 `prismview`를 넣었을 때 실패한 이유도 함께.

해설 `"5432:5432"`는 **"맥의 5432번 → 컨테이너의 5432번" 연결 규칙**(호스트포트:컨테이너포트)이다. URL의 `localhost:5432`는 **접속할 주소**다. 수집기는 Docker 밖(맥)에서 bootRun으로 실행되므로 맥 입장의 주소인 `localhost`로 접속해야 한다. compose의 서비스 이름은 **같은 Docker 네트워크 안의 컨테이너끼리만** 쓰는 DNS 이름이라, 맥에서는 해석되지 않는다. 나중에 수집기도 컨테이너로 띄우면 그때는 서비스 이름이 맞는 호스트가 된다.

6.3 HikariCP — 커넥션 풀

DB 연결은 만들 때 비싸다(TCP + 인증). 그래서 미리 몇 개를 열어 두고 빌려 쓰고 반납한다. SQLAlchemy engine의 `pool_size`, `max_overflow`가 하던 일을 HikariCP가 한다. 로그의 `HikariPool-1 - Start completed`는 "풀을 만들고 실제 연결 하나에 성공했다"는 의미다.

6.4 Flyway — Alembic과 무엇이 다른가

	Alembic	Flyway
마이그레이션 작성	Python 코드 (`op.create_table`) / autogenerate	**순수 SQL 파일** (자동 생성 없음)
파일 이름	랜덤 revision id + down_revision 체인	`V1__create_candle_30s.sql` — **버전 번호 순서**
기록 테이블	`alembic_version` (현재 버전 1행)	`flyway_schema_history` (적용 이력 전체 + 체크섬)
실행 시점	`alembic upgrade head` 를 따로 실행	Spring Boot에선 **앱 시작 시 자동 실행**
되돌리기	downgrade	(무료판 기준) 되돌리기 대신 새 버전으로 수정

Q 6-3 · 생각해 보기

Flyway는 적용한 파일의 체크섬을 기록한다. 이미 적용된 `V1__...sql`을 열어 컬럼 하나를 고치고 다시 실행하면 어떻게 될까? 그럼 스키마 변경은 어떻게 해야 할까?

해설 기록된 체크섬과 현재 파일이 달라 **검증 실패로 앱 시작이 중단**된다. "이미 운영 DB에 적용된 이력이 바뀌었다"는 건 환경마다 스키마가 달라질 수 있다는 위험 신호이기 때문이다. 변경은 항상 `V2__add_xxx.sql`처럼 **새 버전 파일을 추가**한다. 적용된 마이그레이션은 불변 기록으로 다룬다 — 감사 추적을 중시하는 금융권 문화와도 맞닿는 지점이다.

6.5 로그로 읽는 실제 실행 #3

실제로 겪은 일 #3 · local 프로필로 실행 → 연결 성공 → 바로 종료

```
The following 1 profile is active: "local"
HikariPool-1 - Start completed.                          ← DB 연결 성공
Creating Schema History table "public"."flyway_schema_history" ...
No migrations found. Are your locations set up correctly?   ← 경고
Started CollectorApplication in 1.x seconds
(프로세스 종료)
```

두 가지를 읽어야 한다.

① **"No migrations found"** — 에러가 아니라 경고다. `src/main/resources/db/migration/` 에 SQL 파일이 아직 없을 뿐. 이력 테이블은 미리 만들어 두었다.

② **바로 종료** — 고장이 아니다. 아래 Q로 생각해 보자.

Q 6-4 · 생각해 보기

FastAPI 앱은 `uvicorn`으로 띄우면 계속 떠 있다. Spring Boot 앱은 "Started..." 직후 종료됐다. 무엇이 앱을 살아 있게 만드는가?

해설 JVM은 **살아 있는 (데몬이 아닌) 스레드가 하나도 없으면** 종료된다. uvicorn이나 Spring Web의 내장 톰캣은 요청을 기다리는 스레드를 계속 띄워 두기 때문에 프로세스가 살아 있다. 우리는 Web 의존성을 넣지 않았으므로 기다리는 스레드가 없다 → main이 끝나자 종료. 수집 루프를 `@Scheduled` 스케줄러나 WebSocket 수신 스레드 등으로 돌리기 시작하면 그 스레드가 앱을 살려 둔다. 지금의 종료는 **"할 일이 없어서 정상 종료"**다.

CHAPTER 06 · 한 장 요약

- JDBC = DB-API, HikariCP = engine의 풀, JdbcTemplate = SQLAlchemy Core, Flyway = Alembic.
- URL은 `jdbc:` 로 시작. Docker 밖에서 실행하면 호스트는 `localhost` .
- Flyway는 순수 SQL, 버전 순서, 체크섬 검증, **앱 시작 시 자동 실행**. 적용된 파일은 고치지 말 것.
- "No migrations found"는 경고, 바로 종료는 살아 있는 스레드가 없어서.

CHAPTER 07

테스트: @SpringBootTest와 프로필

pytest의 fixture와 conftest에 대응하는 것이 어노테이션으로 표현된다. 지금 막혀 있는 #4를 해결한다.

7.1 기본 테스트 읽기

```
@SpringBootTest                          // "진짜 앱처럼 ApplicationContext 전체를 띄워라"
class CollectorApplicationTests {
    @Test                                // JUnit 5 테스트 메서드 = pytest의 test_ 함수
    void contextLoads() { }              // 본문이 비어 있다!
}
```

Q 7-1 · 생각해 보기

본문이 비어 있는 테스트가 실패할 수 있는 이유는? 이 테스트는 무엇을 검증하고 있는가?

해설 테스트 메서드를 실행하기 **전에** @SpringBootTest가 앱 전체(4장의 ①~④)를 띄운다. Bean 하나라도 만들지 못하면 메서드에 도달하기 전에 실패한다. 즉 contextLoads는 "**이 설정으로 앱이 시작될 수 있는가**"를 검증하는 스모크 테스트다. pytest로 치면 `TestClient(app)`을 만드는 fixture에서 이미 터지는 것과 같다.

7.2 #4 — 테스트에도 프로필이 없다

실제로 겪은 일 **#4** · bootRun은 local로 해결, 테스트는 여전히 실패

`--spring.profiles.active=local` 은 **bootRun 실행 인자**였다. `./gradlew test` 는 그 인자를 받지 않으므로 테스트의 Spring은 여전히 default 프로필 → #2와 같은 원인으로 DataSource 생성 실패. 두 실행 경로는 **설정 출처가 서로 독립**이라는 점을 기억하자.

```
import org.springframework.test.context.ActiveProfiles;

@SpringBootTest
@ActiveProfiles("local")                 // 이 테스트 클래스의 컨텍스트는 local 프로필로
class CollectorApplicationTests {
    @Test void contextLoads() { }
}
```

이렇게 하면 테스트가 `application-local.yml`을 읽고 Docker의 PostgreSQL에 실제로 연결한다. 단, **docker compose up 상태여야** 통과한다.

pytest	JUnit 5 + Spring Test
`def test_x():`	`@Test void x()`
`assert a == b`	`assertEquals(b, a)` 또는 AssertJ `assertThat(a).isEqualTo(b)`
`@pytest.fixture`	`@BeforeEach`, 또는 Bean 주입
`monkeypatch.setenv`	`@ActiveProfiles`, `@TestPropertySource`, `@DynamicPropertySource`
`app.dependency_overrides`	테스트용 Bean 교체 (`@MockitoBean` 등)

Q 7-2 · 생각해 보기

`@ActiveProfiles("local")` 해법의 약점은 무엇일까? GitHub Actions 같은 CI에서 이 테스트를 돌린다고 상상해 보라.

해설 테스트가 **"내 맥에 Docker DB가 떠 있다"는 외부 상태에 의존**한다. CI 서버엔 그 DB가 없으니 실패하고, 로컬에서도 DB 데이터가 테스트마다 남아 결과가 달라질 수 있다. 또 "로컬 개발 설정"과 "테스트 설정"이 한 프로필에 섞인다. 지금 단계에선 충분히 좋은 해법이지만, 이 약점을 해결하는 것이 다음의 Testcontainers다.

7.3 예고: Testcontainers

테스트가 시작될 때 **일회용 PostgreSQL 컨테이너를 직접 띄우고**, 그 주소를 Spring 설정에 주입한 뒤, 끝나면 폐기한다. pytest에서 `testcontainers-python`으로 fixture를 만들던 것과 같은 아이디어다.

```
@SpringBootTest
@Testcontainers
class CandleRepositoryTest {
    @Container
    static PostgreSQLContainer<?> pg = new PostgreSQLContainer<>("postgres:17");

    @DynamicPropertySource                // 실행 중에 결정된 주소를 설정에 주입
    static void props(DynamicPropertyRegistry r) {
        r.add("DB_URL", pg::getJdbcUrl);
        r.add("DB_USERNAME", pg::getUsername);
        r.add("DB_PASSWORD", pg::getPassword);
    }
}
```

여기서 5장의 설계가 빛난다. 공통 yml이 `${DB_URL}`만 요구하므로, 테스트는 **그 키만 채워 주면** 된다. 그리고 앱이 시작될 때 Flyway가 V1을 자동 적용하므로 테이블도 실제와 똑같이 준비된다. (Spring Boot 버전에 따라 `@ServiceConnection`으로 더 간단히 연결하는 방법도 있다. 세부 문법은 그 시점의 공식 문서로 확인할 것.)

CHAPTER 07 · 한 장 요약

- @SpringBootTest는 앱 전체를 띄운다. 빈 contextLoads = "시작 가능한가" 스모크 테스트.
- bootRun 인자는 테스트에 전달되지 않는다 → `@ActiveProfiles("local")`.
- 약점: 외부 DB 상태 의존 → Testcontainers + `@DynamicPropertySource` 로 해결 예정.

CHAPTER 08

다음 단계 지도

지금까지의 개념이 앞으로 작성할 코드의 어디에 쓰이는지 미리 연결해 둔다. 정답 코드라기보다 "설계할 때 던질 질문" 위주.



8.1 ① Flyway V1 — candle_30s

```
-- src/main/resources/db/migration/V1__create_candle_30s.sql  (밑줄 2개!)
CREATE TABLE candle_30s (
    inst_id   TEXT          NOT NULL,      -- 예: BTC-USDT-SWAP
    ts        TIMESTAMPTZ   NOT NULL,      -- 30초 구간 시작 시각
    open      NUMERIC(20,8) NOT NULL,
    high      NUMERIC(20,8) NOT NULL,
    low       NUMERIC(20,8) NOT NULL,
    close     NUMERIC(20,8) NOT NULL,
    volume    NUMERIC(28,8) NOT NULL,
    PRIMARY KEY (inst_id, ts)
);
```

(정밀도·컬럼은 예시 — OKX 응답 필드와 맞춰 확정할 것)

Q 8-1 · 생각해 보기

왜 `id BIGSERIAL` 같은 인공 키 대신 `(inst_id, ts)` 복합 기본키를 쓸까? 수집기가 재시작되어 같은 30초봉을 다시 받는 상황을 떠올려 보라.

해설 한 종목의 한 30초 구간은 세상에 **단 하나**이므로 그 자체가 자연스러운 식별자다. 이 키에 UNIQUE 제약이 있어야 `ON CONFLICT (inst_id, ts) DO UPDATE`로 "재수신 시 덮어쓰기"가 가능하고, 중복 행이 원천 차단된다(멱등성). 또 백테스트가 "BTC의 특정 기간"을 조회하는 패턴과 인덱스 순서가 일치한다. 인공 키를 쓰면 같은 캔들이 두 번 들어가는 걸 DB가 막아 주지 못한다.

8.2 ② Candle record — 타입 선택

필드	Java 타입	이유
instId	`String`	—
ts	`Instant` 또는 `long`	`Instant`는 UTC 시각을 뜻이 명확하게 표현. OKX는 밀리초 문자열을 주므로 변환 지점을 한 곳에 모을 것
OHLC·volume	`BigDecimal`	2.2의 부동소수 문제. OKX가 숫자를 **문자열**로 주는 것도 정밀도를 지키기 위해서다

Q 8-2 · 생각해 보기

`BigDecimal` 두 값이 같은지 비교할 때 `a.equals(b)`를 쓰면 함정이 있다. `new BigDecimal("1.0")`과 `new BigDecimal("1.00")`은 equals로 같을까?

해설 **다르다(false).** BigDecimal의 equals는 값뿐 아니라 **scale(소수점 자리수)**까지 비교한다. 값만 비교하려면 `a.compareTo(b) == 0`을 쓴다. 2.6의 compact 생성자에서 `high.compareTo(low)`를 쓴 이유다. record의 자동 equals도 필드의 equals를 쓰므로, 테스트에서 Candle을 비교할 때 이 차이를 의식해야 한다 (DB에서 읽은 값은 NUMERIC(20,8)의 scale로 돌아온다).

8.3 ③ RestClient — httpx 대응

PYTHON · httpx

```python
client = httpx.Client(
    base_url="https://www.okx.com")
r = client.get("/api/v5/market/candles",
    params={"instId": inst, "bar": "1s"})
r.raise_for_status()
data = r.json()
```

JAVA · RestClient

```java
RestClient client = RestClient.builder()
    .baseUrl("https://www.okx.com").build();

OkxResponse res = client.get()
    .uri(u -> u.path("/api/v5/market/candles")
        .queryParam("instId", inst)
        .queryParam("bar", "1s").build())
    .retrieve()            // 4xx/5xx면 예외
    .body(OkxResponse.class); // JSON → 객체
```

`.body(OkxResponse.class)`는 pydantic의 `Model.model_validate(r.json())`와 같은 역할 — 응답 JSON을 **타입이 있는 record**로 변환한다(Jackson 라이브러리). RestClient 자체도 4장 방식대로 `@Bean`으로 등록해 서비스에 주입받는 것이 정석이다. (사용할 OKX 엔드포인트·파라미터는 공식 API 문서로 확인 후 확정)

Q 8-3 · 생각해 보기

`u -> u.path(...)...`는 무엇일까? Python 문법 중 가장 가까운 것은?

해설 **람다식**이다. `lambda u: u.path(...)`와 같다. "URI를 만드는 방법"을 함수로 넘긴다. 7.3의 `pg::getJdbcUrl`은 **메서드 참조**로, Python에서 `pg.get_jdbc_url`을 괄호 없이 함수 자체로 넘기는 것과 같다. Java에선 이런 함수 값이 들어갈 자리의 타입이 "함수형 인터페이스"(메서드가 하나인 interface)로 정해져 있다.

8.4 ④ JdbcTemplate 배치 upsert

```
@Repository
public class JdbcCandleRepository implements CandleRepository {
    private final JdbcTemplate jdbc;               // 자동 설정이 만든 Bean을 주입받음
    public JdbcCandleRepository(JdbcTemplate jdbc) { this.jdbc = jdbc; }

    private static final String UPSERT = """
        INSERT INTO candle_30s (inst_id, ts, open, high, low, close, volume)
        VALUES (?, ?, ?, ?, ?, ?, ?)
        ON CONFLICT (inst_id, ts) DO UPDATE SET
          high = EXCLUDED.high, low = EXCLUDED.low,
          close = EXCLUDED.close, volume = EXCLUDED.volume
        """;                                          // """ 텍스트 블록 = Python의 삼중 따옴표

    @Override
    public void upsertAll(List<Candle> candles) {
        jdbc.batchUpdate(UPSERT, candles, 500, (ps, c) -> {   // 500개씩 묶어서
            ps.setString(1, c.instId());
            ps.setTimestamp(2, Timestamp.from(c.ts()));
            ps.setBigDecimal(3, c.open());  /* ... 7번까지 */
        });
    }
}
```

`?`는 SQLAlchemy의 `:name` 바인드 파라미터와 같고, **SQL 인젝션을 막는** 같은 원리다. 이름으로 바인딩하고 싶으면 `NamedParameterJdbcTemplate`을 쓴다.

Q 8-4 · 생각해 보기

UPDATE 절에서 `open`은 갱신하지 않았다. 의도적인 선택일까? 30초봉이 "아직 진행 중일 때 한 번, 완성 후 한 번" 저장되는 경우를 생각해 보라.

해설 의도적이다. 한 30초 구간의 **시가(open)는 구간의 첫 체결**로 처음부터 확정되지만, 고가·저가·종가·거래량은 구간이 끝날 때까지 바뀐다. 그래서 갱신 대상에서 open을 빼는 설계가 가능하다. 다만 "완성된 봉만 저장"하기로 결정하면 UPDATE 범위는 덜 중요해진다. 어느 쪽이든, **이 결정과 이유를 이슈나 커밋 메시지에 남겨 두면** 면접에서 설명할 좋은 재료가 된다. 그리고 `reWriteBatchedInserts=true`(6.2)가 이 batchUpdate를 multi-row INSERT로 바꿔 빠르게 만든다.

Q 8-5 · 종합

이 노트 전체를 바탕으로, `./gradlew bootRun --args='--spring.profiles.active=local'`을 실행했을 때 V1 마이그레이션이 적용되고 수집이 시작되기까지의 과정을 순서대로 말해 보라. (면접에서 "Spring Boot 앱이 시작될 때 무슨 일이 일어나나요?"에 대한 답이 된다.)

해설 ① gradlew가 지정 버전의 Gradle로 컴파일 후 main 실행, 인자가 `String[] args`로 전달 → ② SpringApplication이 인자·환경변수·`application.yml`·`application-local.yml`을 합쳐 Environment 구성, `${DB_URL}` 해석 → ③ 컴포넌트 스캔으로 내 @Service·@Repository 발견, 자동 설정이 DataSource(HikariCP)·JdbcTemplate·Flyway Bean 준비 → ④ Bean 생성 중 Flyway가 `db/migration`의 V1을 확인해 미적용분 실행, 이력 테이블에 기록 → ⑤ 내 서비스 Bean에 JdbcTemplate·RestClient가 생성자 주입 → ⑥ 스케줄러 등 수집 스레드 시작, 그 스레드가 JVM을 살아 있게 유지. 어느 단계든 실패하면 시작 자체가 중단된다(fail-fast).

이 노트의 한 줄

- Java는 **미리 검사하고**(컴파일), Spring은 **미리 조립한다**(시작 시 Bean 생성). 지금까지의 에러 4개는 모두 "조립 단계에서 재료(설정)가 없었다"는 같은 이야기였다.